서울시립대학교
UNIVERSITY OF SEOUL

전자전기컴퓨터설계실험II

실험 전 보고서

STM32 인터럽트·타이머·PWM 및 FPGA 개별 실습

학과	전자전기컴퓨터공학부
이름 / 학번	조성현 / 2023440129
작성일	2026년 10월 19일
대상	버튼 인터럽트 · 주기 타이머 · PWM 제공 예제 · STM32 과제3 · STM32 과제4 · FPGA 피아노 · 신호등 · ADC 실습
검증환경	Windows 11 / VS Code / STM32 CMake / VaporView / AMD Vivado 2026.1 XSim

소스 식별

GitHub 계정: a40093632-ai

GitHub 저장소: https://github.com/a40093632-ai/ece2-lab5

제출 태그: lab5-pre-v1

커밋 해시: c2faabbe6e325799f048eecd532b96f4930ff8fd

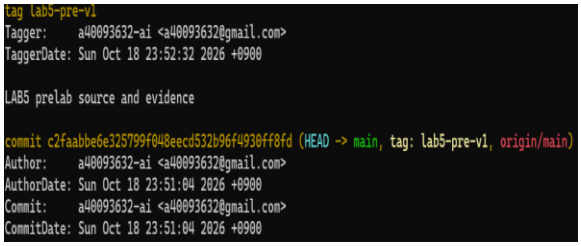


그림 1: GitHub 제출 태그 및 기준 커밋 확인

1. 실험 목적 및 범위

실험 목적

본 실험의 목적은 STM32에서 외부 인터럽트와 주기 타이머 인터럽트를 이용하여 입력과 시간에 따른 동작을 처리하고, 타이머의 PWM 기능으로 출력 신호의 듀티비를 제어하는 것이다. 제공 예제를 통해 버튼 인터럽트 처리 함수, TIM6의 주기적 인터럽트, TIM3의 PWM 출력 방법을 확인한다.

STM32 과제3에서는 LAB4의 LED 간격 변경 기능을 버튼 인터럽트와 TIM6 인터럽트로 다시 구현한다. 과제4에서는 버튼 입력마다 PB4의 PWM 듀티비가 25%, 50%, 75% 순서로 순환하도록 구현하고, 디버거를 이용하여 변수와 실행 흐름을 확인하는 방법을 준비한다.

FPGA 개별 실습에서는 피아노, 신호등 제어, ADC 직렬 송수신 회로를 순서대로 확인한다. 각 실습은 제공된 RTL과 테스트벤치를 이용하여 입력에 따른 출력, 상태 전환 및 직렬 송수신 동작을 검사한다. PASS 메시지와 대표 파형을 확보하고, 실제 보드에서 확인할 동작을 정리한다.

실험 범위

	실습 내용	사전 검증 방법
예제1	버튼 입력에 따른 PA5 LED 반전	CMake 빌드
예제2	TIM6 인터럽트 10회마다 PA5 LED 반전	CMake 빌드
예제3	PB4에서 1kHz, 50% PWM 출력	CMake 빌드
과제3	인터럽트로 LED 상태 변경 간격 1-2-4-8-1초 제어	CMake 빌드
과제4	버튼으로 PWM 듀티비 25-50-75-25 제어	CMake 빌드
실습1	피아노 음 선택 및 구형파 생성	Verilog 시뮬레이션
실습2	차량, 보행자 신호와 비상 점멸 제어	Verilog 시뮬레이션
실습3	ADC 명령 전송 및 직렬 데이터 수신	Verilog 시뮬레이션

2. 관련 이론

인터럽트와 우선순위

인터럽트는 외부 입력이나 주변장치의 사건이 발생했을 때 CPU가 실행 중인 흐름을 잠시 중단하고, 해당 인터럽트 서비스 루틴(ISR)을 실행하는 방식이다. 처리가 끝나면 이전 실행 흐름으로 복귀한다. 따라서 메인 루프에서 입력을 반복해서 검사하지 않아도 사건에 대응할 수 있다.[1]

우선순위 숫자가 작을수록 높은 우선순위를 나타내며, 높은 선점 우선순위의 인터럽트는 낮은 우선순위의 ISR 실행 중에도 먼저 처리될 수 있다.[1] 본 실험에서는 TIM6의 우선순위가 4, 버튼의 우선순위가 7이므로 TIM6가 버튼 ISR을 선점할 수 있다. 따라서 ISR은 필요한 작업만 짧게 수행하고, 여러 ISR이 함께 사용하는 상태를 변경할 때는 실행 순서를 고려해야 한다.

타이머의 분주와 주기

타이머는 일정한 클록을 계수하여 시간 간격을 만든다. 클록 주파수와 한 주기에 필요한 계수 횟수를 알면 주기를 계산할 수 있으며, 이를 이용하여 주기적으로 인터럽트를 발생시킬 수 있다.[1]본 실험의 STM32 타이머는 입력 클록을 프리스케일러(PSC)로 분주한 뒤, 상향 카운터가 0부터 자동 재장전 값(ARR)까지 계수한다. 이 설정에서 반복 계수가 없는 경우 갱신 주기는 다음과 같다.

$$f_{CNT} = \frac{f_{TIM}}{PSC + 1}$$

$$T_{update} = \frac{ARR + 1}{f_{CNT}} = \frac{(PSC + 1)(ARR + 1)}{f_{TIM}}$$

3. 개발 및 검증 환경

개발환경

항목	사용 환경
운영체제	Windows 11 64-bit
편집기	VS Code 1.137.0
파형 확인	VaporView
FPGA Part	Xc7s75fgga484-1
FPGA 도구	AMD Vivado 2026.1
보드	Combo II-DLD S75
STM32 빌드	CMake 기반 STM32
STM32 대상	NUCLEO-G474RE

주요 핀

핀	용도
PA5	보드 내장 LED 출력

PB4	TIM3_CH1 PWM 출력
PC13	사용자 버튼 인터럽트

4. 버튼 인터럽트 제공 예제1

실습 내용

PC13 버튼에 의한 외부 인터럽트가 발생하면 HAL 콜백을 거쳐 button_isr_p7()이 호출된다. 함수에서는 마지막으로 인정한 입력 이후 200ms 이상 경과했는지 검사하고, 조건이 만족되면 마지막 입력 시각을 갱신한 뒤 PA5 LED를 한 번 반전한다.

동작 흐름

1. PC13 버튼의 외부 인터럽트 발생시 button_isr_p7() 이 호출된다.

2. HAL_GetTick()으로 현재 시각을 읽는다.

3. 마지막으로 인정한 입력 이후 200ms 이상 지났는지 검사한다.

4. 조건을 만족하면 마지막 입력 시각을 갱신하고 PA5 LED를 반전한다.

5. 조건을 만족하지 않으면 LED 상태를 유지한다.

예상 결과

입력	예상 결과
유효한 버튼 입력 1회	PA5 LED 상태 1회 반전
200ms 이내 추가 인터럽트	LED 상태 유지
200ms 이상 지난 뒤 새 입력	PA5 LED 다시 반전

사전 검증 결과

제공된 버튼 인터럽트 코드를 적용하여 CMake 빌드를 수행하였다. 프로그램이 정상적으로 컴파일·링크되었으며 빌드 성공을 확인하였다. 실제 버튼 입력과 LED 반응은 보드 실험에서 확인한다.

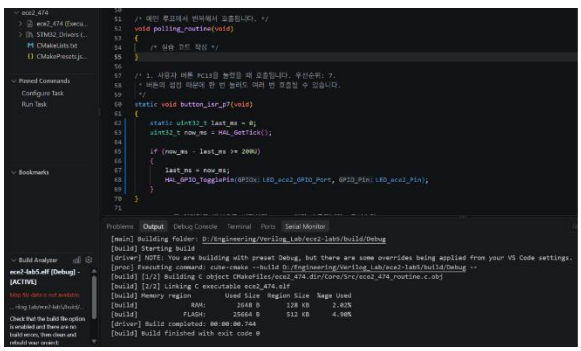


그림 2: 버튼 인터럽트 예제의 CMake 빌드 결과

링크: 버튼 인터럽트 예제코드

5. 주기 타이머 제공 예제2

실습 내용

ece2_474_init()에서 TIM6 인터럽트를 시작하면 100ms마다 tim6_isr_p4()가 호출된다. 함수 내부의 정적 변수로 호출 횟수를 저장하고, 10회에 도달하면 횟수를 0으로 초기화한 뒤 PA5 LED를 반전한다.

동작 흐름

1. 초기화 함수에서 HAL_TIM_Base_Start_IT(&htim6)로 TIM6 인터럽트를 시작한다.

2. 100ms마다 tim6_isr_p4()가 호출된다.

3. 인터럽트가 호출될 때마다 count를 1 증가시킨다.

4. count가 10에 도달하면 0으로 초기화하고 PA5 LED를 반전한다.

예상 결과

TIM6 인터럽트 10회는 100ms x 10 = 1s이므로 LED는 1초마다 상태가 바뀐다. ON과 OFF를 모두 포함한 완전한 반복 주기는 2초이다.

사전 검증 결과

TIM6 시작 코드와 인터럽트 계수 코드를 적용한 상태에서 CMake 빌드가 성공하였다. 실제 인터럽트 발생 간격과 LED의 상태 변경 간격은 보드에서 확인한다.

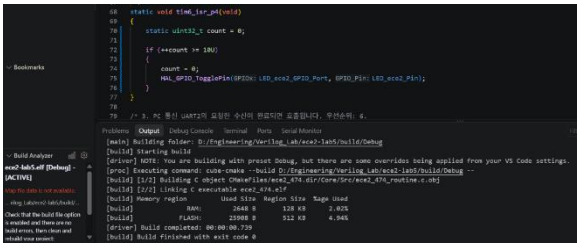


그림 3: TIM6 주기 타이머 예제의 CMake 빌드 결과

링크: 주기 타이머 예제코드

6. PWM 출력 제공 예제3

실습 내용

TIM3 채널1의 PWM을 시작하고 비교 값을 500으로 설정하여 PB4에서 1kHz, 50% 듀티비의 파형을 출력하도록 구성한다. PB4는 내장 LED가 연결된 PA5와 다른 핀이므로,

출력 확인에는 PB4와 GND에 연결한 계측기를 사용한다.

동작 흐름

1. 초기화 함수에서 TIM3 채널1의 PWM 출력을 시작한다.

2. 비교 값을 500으로 설정한다.

3. TIM3가 PB4에서 1kHz, 50% 듀티비의 PWM을 생성한다.

4. 초기화 이후에는 타이머 하드웨어가 PWM 출력을 반복한다.

예상 결과

PB4에서 주기 1ms, HIGH 시간 0.5ms, LOW 시간 0.5ms의 PWM이 출력될 것으로 예상한다.

사전 검증 결과

TIM3 채널1의 PWM 시작과 비교 값 설정 코드를 적용한 상태에서 CMake 빌드가 성공하였다. PB4의 실제 출력 주파수와 듀티비는 보드에 프로그램을 다운로드한 뒤 계측기로 확인한다.

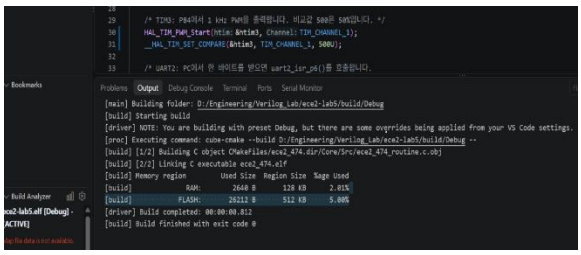


그림 4: PWM 출력 예제의 CMake 빌드 결과

링크: PWM 출력 예제코드

7. STM32 과제 3

설계 목표

LAB4에서 구현한 LED 간격 변경 기능을 버튼 인터럽트와 TIM6 인터럽트로 재현한다. 버튼을 누를 때마다 PA5 LED의 상태 변경 간격이 1초, 2초, 4초, 8초, 1초 순서로 순환하도록 한다. 버튼 확인과 LED 시간 제어는 ISR에서 수행한다.

핵심 설계

100ms마다 발생하는 TIM6 인터럽트를 elapsed_ticks로 계수하고, interval_ticks에 도달하면 PA5 LED를 반전한다. 버튼 ISR에서는 200ms 디바운스를 적용한 뒤 interval_ticks를 10→20→40→80→10으로 변경한다. 간격 변경 시 경과 횟수와 TIM6 카운터·갱신 플래그를 초기화하며, 이 작업은 인터럽트를 일시적으로 차단한 짧은 임계 구역에서 수행한다. 공유 변수에는 volatile을 적용하였다.

```
uint32_t primask = __get_PRIMASK();
__disable_irq();

interval_ticks = (interval_ticks >= 80U)
                 ? 10U : interval_ticks * 2U;

elapsed_ticks = 0U;
__HAL_TIM_SET_COUNTER(&htim6, 0U);
__HAL_TIM_CLEAR_FLAG(&htim6, TIM_FLAG_UPDATE);

__set_PRIMASK(priMask: primask);
```

그 림 5 : S T M 3 2 과 제 3 핵 심 코 드

예상 결과

버튼 입력 횟수	Interval_ticks	LED 상태 변경 간격	ON/OFF 반복 주기
초기	10	1초	2초
1회	20	2초	4초
2회	40	4초	8초
3회	80	8초	16초
4회	10	1초	2초

사전 검증 결과

버튼 인터럽트, 공유 변수, 임계 구역 및 TIM6 계수 로직을 포함한 과제3 프로그램이 정상적으로 컴파일·링크되었다. 실제 PA5 출력과 버튼 입력에 따른 간격 변경은 보드에서 확인한다.

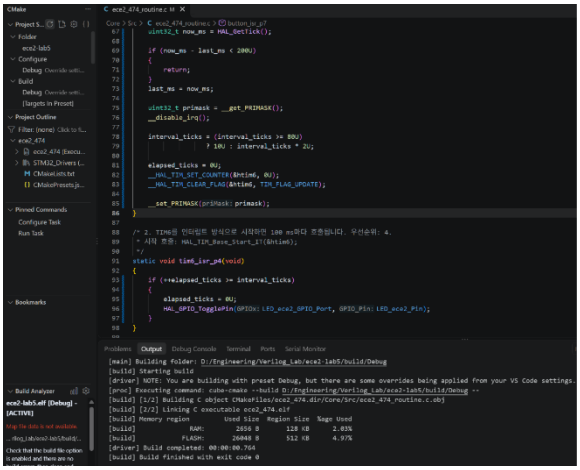


그림 6: STM32 과제 3 코드 및 CMake 빌드 결과

링크: 과제3 전체 코드

링크: 과제3 빌드 및 핵심 코드 증빙

8. STM32 과제 4

설계 목표

PC13 버튼 입력마다 PB4의 PWM 듀티비가 25%, 50%, 75%, 25% 순서로 순환하도록 구현한다. PWM 주파수는 1kHz로 유지하고, 디버거의 중단점과 단계 실행을 통해 듀티비 변수 및 비교 값의 변화를 확인한다.

핵심 설계

초기 듀티비를 25%로 설정하고, 비교 값을 지정한 뒤 TIM3 채널1 PWM을 시작한다. 버튼 ISR에서는 200ms 디바운스를 적용한 뒤 duty_percent를 25→50→75→25로 변경한다. 비교값을 duty_percent×10으로 갱신하여 1kHz 주파수는 유지하면서 PB4의 HIGH 시간만 변경한다.

```
duty_percent = (duty_percent >= 75U)
             ? 25U : duty_percent + 25U;

__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, duty_percent * 10U);
```

그 림 7 : S T M 3 2 과 제 4 핵 심 코 드

예상 결과

버튼 입력 횟수	Duty_percent	비교값	HIGH 시간
초기	25	250	0.25ms
1회	50	500	0.50ms
2회	75	750	0.75ms
3회	25	250	0.25ms

사전 검증 결과

PWM 초기화 및 듀티비 변경 코드를 포함한 과제4 프로그램이 정상적으로 컴파일·링크되었다. 실제 1kHz 출력과 세 단계의 HIGH 시간, 중단점 및 변수 확인은 보드 실험에서 수행한다.

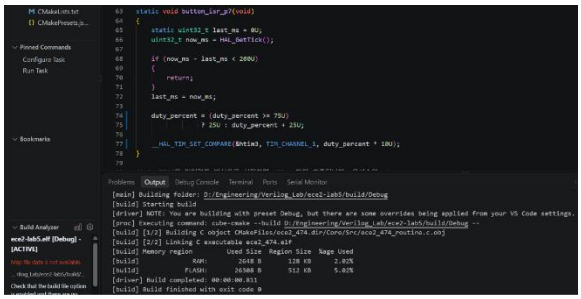


그림 8: STM32 과제 4 코드 및 CMake 빌드 결과

링크: 과제4 전체 코드

링크: 과제4 빌드 및 핵심 코드 증빙

9. FPGA 실습 1 · 피아노

설계 목표

버튼 입력에 따라 여덟 음 중 하나를 선택하고 피에조에 출력할 구형파를 생성한다. note_select는 버튼을 주파수 값으로 변환하며, square_tone은 선택된 주파수의 반주기를 계수하여 출력을 반전한다. 최상위 piano는 두 모듈을 연결한다.

핵심 설계

note_select는 키 입력을 음별 주파수로 변환하고, square_tone은 CLK_HZ/(2×frequency_hz)로 구한 반주기 계수 값에 따라 출력을 반전한다. 무입력·복수 입력에서는 무음으로 처리한다. 리셋이나 음 변경 시 계수기와 출력을 초기화하여 이전 음의 계수 상태가 이어지지 않도록 한다.

```
wire [31:0] half_cycles = frequency_hz == 0 ? 1 :
                          CLK_HZ / (2 * frequency_hz);

always @(posedge clk_50mhz) begin
    if(rst_p) begin
        count<=0;
        tone<=0;
        previous_hz<=0;
    end
    else if(frequency_hz==0 || frequency_hz!=previous_hz) begin
        count<=0;
        tone<=0;
        previous_hz<=frequency_hz;
    end
    else if(count >= half_cycles-1) begin
        count<=0;
        tone<=~tone;
    end
    else count<=count+1;
end
```

그 림 9 : 피 아 노 구 형 파 생 성 핵 심 코 드

테스트벤치 검증 방법

테스트벤치는 여덟 키를 순서대로 입력하고 주파수 선택 값을 검사한다. 각 음에 대해 예상 반주기를 계산한 뒤 매 클록의 piezo 출력이 예상 반전 시점과 일치하는지 확인한다.

시뮬레이션 시간을 줄이기 위해 DUT의 CLK_HZ를 10000으로 설정하고 입력 클록은 10ns 주기로 생성한다. 따라서 이 시뮬레이션의 ns 시간축은 실제 보드의 음 주기와 직접 같지 않으며, 계수와 출력 반전 관계를 검사하는 데 사용한다.

예상 결과

Key(16진수)	선택 음	주파수 설정값
80	도	262Hz
40	레	294Hz
20	미	330Hz
10	파	349Hz

검증 결과

대표 파형에서는 키가 80-40-20-10으로 바뀌며 주파수 설정 값이 106-126-14A-15D로 변한다. 이 값은 16진수이며 각각 10진수로 262, 294, 330, 349에 해당한다. 예상결과와 일치함을 알 수 있고, 각 음 구간에서 출력 반주기는 짧아져 선택 음이 높아질수록 출력 주기가 감소함을 확인하였다.



그림 10: 피아노 Verilog PASS 결과

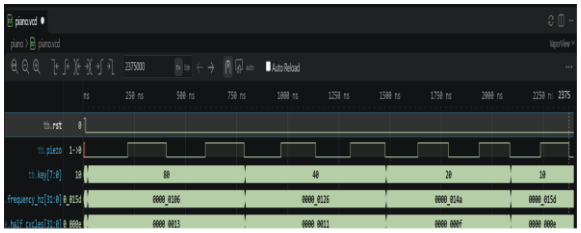


그림 11: 피아노 출력 시뮬레이션 파형

링크: 피아노 전체 소스

링크: 피아노 증빙 자료

10. FPGA 실습 2 · 신호등 제어

설계 목표

네 방향의 차량 신호와 보행자 신호를 순서대로 제어하고, 보행자 녹색 점멸 및 비상 점멸 기능을 확인한다.

second_tick은 시간 기준과 점멸 신호를 생성한다. traffic_sequence는 현재방향 direction과 시간 단계 phase를 저장한다. traffic_lights는 현재 상태를 차량·보행자 LED 출력으로 변환한다.

핵심 설계

traffic_sequence는 시간 기준 펄스마다 phase를 0~12로 증가시키고, 12 다음에는 0으로 되돌리며 다음 방향으로 전환한다. traffic_lights는 선택 방향의 차량 신호를 녹색에서 황색으로 바꾸고, 대응 보행자 신호를 녹색 유지·점멸·적색 순서로 제어한다. flicker=1에서는 차량 황색과 보행자 적색을 점멸시키며, 방향과 단계를 0으로 초기화한다.

```
always @(posedge clk_50mhz) begin
    if(rst_p || flicker) begin
        direction<=0;
        phase<=0;
    end
    else if(tick)
        if(phase==12) begin
            phase<=0;
            direction<=direction+1'b1;
        end
        else
            phase<=phase+1'b1;
end
```

그림 12: 신호등 제어 핵심 코드

테스트벤치 검증 방법

시뮬레이션에서는 CLK_HZ=4로 설정하여 시간을 단축한다. 테스트벤치는 네 방향과 방향별 13개 단계를 순서대로 검사하고, 각 단계의 차량·보행자 LED가 예상 마스크와 일치하는지 확인한다. 이후 비상 점멸과 리셋을 검사한다.

예상 결과

Phase	선택 방향 차량	대응 보행자
0~3	초록	초록 유지
4~10	초록	초록 점멸
11~12	노랑	빨강
12 다음	다음 방향 전환	다음 신호 전환

검증 결과

대표 파형에서 445ns에 차량 녹색 출력은 8에서 0으로, 노란색 출력은 0에서 8로 변한다. 이때 보행자 빨간색 출력은 B에서 F로 변하여 모든 보행자 신호가 빨간색이 된다. 또한 525ns에는 direction이 0에서 1로, phase가 C(12)에서 0으로 바뀐다. 동시에 다음 방향의 차량 녹색은 4, 보행자 녹색은 2가 되어 새로운 방향으로 진행이 시작됨을 확인하였다.



그림 13: 신호등 제어 Verilog PASS 결과

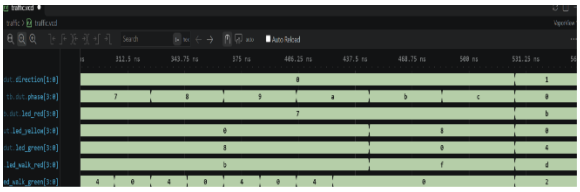


그림 14: 신호등 제어 시뮬레이션 파형

링크: 신호등 제어 전체 소스

링크: 신호등 제어 증빙 자료

11. FPGA 실습 3 · ADC 직렬 송수신

설계 목표

FPGA에서 외부 ADC로 보낼 채널 선택 명령을 생성하고, 직렬로 수신한 데이터를 저장하여 LED에 표시하는 구조를 확인한다. 사전 시뮬레이션에서는 테스트벤치가 ADC의 응답을 모사하며, 명령 전송과 수신 값 저장이 올바른지 검사한다.

핵심 설계

adc_command는 채널 선택 값을 포함하는 16비트 명령을

생성한다. serial_frame16은 cs_n을 LOW로 내려 전송을 시작하고, 직렬 클록의 하강 에지에서 송신 비트를 출력하며 상승 에지에서 수신 비트를 저장한다. 16비트 수신을 마치면 완료 신호를 발생시키고, 최상위 adc는 수신값의 하위 12비트를 sample에 저장한다. LED에는 sample[11:4]를 출력한다.

```
else if(count==HALF_CYCLES-1) begin
    count<=0;
    if(sclk) begin
        sclk<=0;
        mosi<=tx_latched[bit_index];
    end
    else begin
        sclk<=1;
        rx_word<={rx_word[14:0],miso};
        if(bit_index==0) begin
            busy<=0; cs_n<=1; done<=1;
        end
        else bit_index<=bit_index-1'b1;
    end
end
else count<=count+1;
```

그림 15: ADC 직렬 송수신 핵심 코드

테스트벤치 검증 방법

테스트벤치는 ADC 응답으로 16'hA5C3을 직렬 입력하고, FPGA가 전송한 명령을 수집한다. 채널을 2→5→7로 변경하며 각 프레임의 수신 비트 수가 16인지, 채널 명령이 올바른지, 저장 값과 LED 출력이 예상 값과 일치하는지 검사한다. 초기 리셋과 데이터 수신 후 리셋도 확인한다.

예상 결과

선택 채널	예상 송신 명령	예상 sample	예상 LED
2	AB70	5C3	5C
5	B770	5C3	5C
7	BF70	5C3	5C

검증 결과

시뮬레이션에서 "PASS ADC: 3 serial frames, command bits, sample, reset"을 확인하였다. 대표 파형에서는 35ns에 cs_n이 LOW가 되어 첫 프레임이 시작되고, 675ns에 HIGH로 복귀한다. 685ns에는 valid가 HIGH가 되며 sample은 000→5C3, LED는 00→5C로 변경된다. 따라서 테스트벤치가 제공한 직렬 응답의 수신과 저장이 예상대로 수행됨을 확인하였다.

```
## run all
PASS ADC: 3 serial frames, command bits, sample, reset
$finish called at time : 2016 ns : File "D:/Engineering/Verilog_Lab/ece2-lab5/fpga/adc/tb.sv" Line 69
## close_vcd
## quit
INFO: [Common 17-206] Exiting xsim at Sun Oct 18 23:24:08 2026...
```

그림 16: ADC 직렬 송수신 Verilog PASS 결과

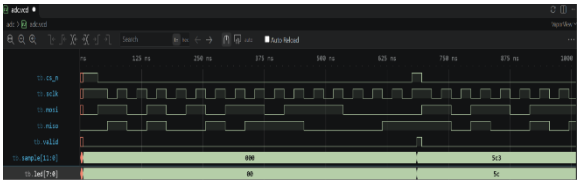


그림 17: ADC 직렬 송수신 시뮬레이션 파형

링크: ADC 직렬 송수신 전체 소스

링크: ADC 직렬 송수신 증빙 자료

12. 사전 검증 결과 및 문제 해결

사전 검증 결과 요약

실습	예상 결과	확인 결과
버튼 인터럽트	유효 버튼 입력마다 LED 반전	빌드 성공
TIM6 예제	100ms 인터럽트 10회마다 LED 반전	빌드 성공
PWM 예제	PB4에서 1kHz · 50% 출력	빌드 성공
STM32 과제3	LED 상태 변경 간격 1-2-4-8-1초	빌드 성공
STM32 과제4	PWM 듀티비 25-50-75-25%	빌드 성공
피아노 실습1	여덟 음·무음·동시 입력·리셋 처리	TB PASS
신호등 실습2	전체 순환·보행자 점멸·비상 점멸·리셋	TB PASS
ADC 실습3	세 프레임의 명령, 수신 값, 리셋 검사	TB PASS

문제 해결 과정

ADC 시뮬레이션에서 VCD 기록을 시작할 때 "compiled without trace information" 오류가 발생하였다. xelab 실행 시 파형 추적 정보를 포함하지 않은 것이 원인이었다. -debug typical 옵션을 추가하여 시뮬레이션 스냅샷을 다시 생성한 뒤 실행하였으며 PASS 메시지와 정상 VCD 생성을 확인하였다.

13. 실험 당일 검증 계획

STM32 실험에서는 과제별 코드를 빌드하고 F5로 보드에 다운로드한다. 초기 중단점에서 정지하면 실행을 재개한 뒤 실제 출력을 확인한다. 과제3에서는 PA5 LED의 상태 변경 간격을 측정하고, 버튼 입력에 따라 1-2-4-8초가 순환하는지 확인한다. 다음으로 과제4에서는 버튼을 누를 때 HIGH 시간이 0.25ms, 0.50ms, 0.75ms로 순환하는지 측정

한다. 중단점과 Watch로 변수 변경을 확인하는 과정은 정상 실행 상태의 파형 측정과 구분한다.

FPGA 실습에서는 각 실습의 RTL과 XDC를 프로젝트에 등록하고 실제 회로 모듈을 Design Top으로 설정한다. 합성, 구현, 비트스트림 생성 및 보드 다운로드를 수행한 뒤 시뮬레이션에서 확인한 동작과 비교한다. 실제 보드 결과의 사진과 영상을 확보하고, 구현 과정에서 확인한 DRC·타이밍 및 프로그램 다운로드 결과를 함께 정리한다.

실습	실험 당일 확인할 항목
피아노	각 키의 음 높이, 무입력·복수 입력의 무음 처리, 리셋
신호등	차량 녹색·황색 순서, 네 방향 순환, 보행자 녹색 유지·점멸·빨강, 비상 점멸
ADC	채널 선택, 아날로그 입력 변화에 따른 LED 값 변화, 리셋

14. 결론

본 실험을 분석하면서 STM32의 외부 인터럽트와 주기 타이머를 이용한 이벤트 처리 방법을 확인하였다. LED 간격 변경 기능을 버튼 ISR과 TIM6 ISR로 구현하고, 공유 상태를 갱신하는 과정에 짧은 임계 구역을 적용하였다. 또한 TIM3의 비교값을 변경하여 PWM 주파수를 유지하면서 듀티비를 순환시키는 프로그램을 작성하였다. 제공 예제와 과제3·4는 모두 정상적으로 빌드되었다.

FPGA에서는 피아노의 음 선택과 구형파 생성, 신호등의 상태 순환과 점멸, ADC의 직렬 송수신을 확인하였다. 세 실습 모두 제공 테스트벤치의 검사를 통과하였으며 대표 파형에서 입력과 출력의 관계를 확인하였다.

현재 사전 검증 범위는 STM32의 컴파일·링크와 FPGA의 디지털 시뮬레이션이다. 실험 당일에는 STM32의 실제 LED 간격과 PWM 파형을 측정하고, FPGA의 장치별 보드 동작을 확인하여 사전 예상 결과와 비교한다.

참고 문헌

[1] J. W. Valvano, *Embedded Systems: Introduction to ARM Cortex-M Microcontrollers*, 2014, ISBN 978-1-4775-0899-2. 관련 공개 학습 자료: J. Valvano and R. Yerraballi, *Embedded Systems—Shape the World*, Chapter 12, "Interrupts," Sections 12.1, 12.3, 12.5. https://users.ece.utexas.edu/~valvano/Volume1/E-Book/C12_Interrupts.htm